COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 08/28/26 | **DATE SUBMITTED:** | 08/28/26 |
| **NAME:** | Prince Benjamin S. Villaruel | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

## ACTIVITY NO. 6 PROGRESS REPORT

### 1. PROCEDURE OUTPUT

What procedures did you perform?

Table 6-1. Data Generated and Observations.

| | |
|---|---|
| **Screenshot** | PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 6> cd "c:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 6\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }<br>19881 20107 30040 19652 19064 20262 27304 29358 1386 23937 12874 13039 12654 31831 12637 22076 19515 27417 16348 20911 16205 21218 29126 31585 4650 16301 31612 27963 4586 14790 13674 2569 4598 4990 745 22736 10768 16163 8292 10850 5524 16687 7354 15663 6141 4800 484 21342 2730 2120 |
| **Observations** | I successfully generated a dataset of 50 random integer values using the srand(time(0)) seeding function and a for loop in C++. By printing the dataset contents to the console, I verified that pseudo-random values were properly generated across the entire array before proceeding to implement the searching algorithms. |

Table 6-2a. Linear Search for Arrays

| | |
|---|---|
| **Screenshot** | **main.cpp**<br>node.h searchingAlgo.h main.cpp<br>main.cpp > ...<br>1 #include <iostream><br>2 #include <cstdlib> //for generating random integers<br>3 #include <time.h> //will be used for our seeding function<br>4 #include "searchingAlgo.h"<br>5 #include "node.h"<br>6<br>7<br>8 const int max_size = 50;<br>9<br>10 int main(){<br>11 int dataset[max_size];<br>12 srand(time(0));<br>13 for(int i = 0; i < max_size; i++){<br>14 dataset[i] = rand();<br>15 std::cout << dataset[i] << " ";<br>16 }<br>17 std::cout << std::endl;<br>18<br>19 std::cout << "What number to find: ";<br>20 int choice;<br>21<br>22 std::cin >> choice;<br>23 |

**searchingAlgo.h**

```h
#include "node.h"

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

template <typename T>
void linearSearchAR(int size, T dataFind,T array[]){

    //while loop
    int i = 0;

    while(i < size){
        if(dataFind == array[i]){
            std::cout<<"Searching is successful\n";
            return;
        }
        i++;
    }
    std::cout<<"Searching is unsuccessful\n";

    //for loopp
    for(int j = 0; j < size; j++){
        if(dataFind == array[j]){
            std::cout<<"Searching is successful\n";
            return;
        }
    }
    std::cout<<"Searching is unsuccessful\n";
}
```

**Output**

```
Villaruel\Activity 6\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
13512 9488 2658 3864 7524 11754 22534 23764 17017 32014 29617 8664 4509 587 9278 1698 23084 13598 4747 8873 244
60 28128 3547 16181 24757 24382 28537 15475 14576 17386 24991 19657 32222 5264 3477 31335 16061 23052 29981 110
39 18663 28634 31503 5750 5531 19149 24385 25083 17664 6148
What number to find: 10
Searching is unsuccessful
Searching is unsuccessful
```

**Observations**

I implemented a linear search function for arrays using both a while loop and a for loop to check each element sequentially against the target value. When testing with the value 10, the algorithm scanned through all 50 randomly generated array elements without finding a match, successfully triggering the searching is unsuccessful output for both loops.

Table 6-2b. Linear Search for Linked List

| Screenshot | main.cpp<br><br>searchingAlgo.h |
|---|---|

```cpp
//create a node
Node<char> *name1 = new_node('P');
Node<char> *name2 = new_node('R');
Node<char> *name3 = new_node('I');
Node<char> *name4 = new_node('N');
Node<char> *name5 = new_node('C');
Node<char> *name6 = new_node('E');

name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = name6;
name6->next = nullptr;

linearSearchLL(name1, 'P');
```

```cpp
template <typename T>
void linearSearchLL(Node<T> *nodes, T dataFind){
    //repeat the loop until the the current node is not null

    while(nodes != nullptr){
        //checks if the item - current node data
        if(nodes->data  == dataFind){
            std::cout << "The data: "<< nodes->data<< " was found successfully.\n";
            return;
        }
        //move on to the next node
        nodes = nodes->next;
    }
    //display unsucessfull
    std::cout << "Searching is unsucessful.\n";
}
```

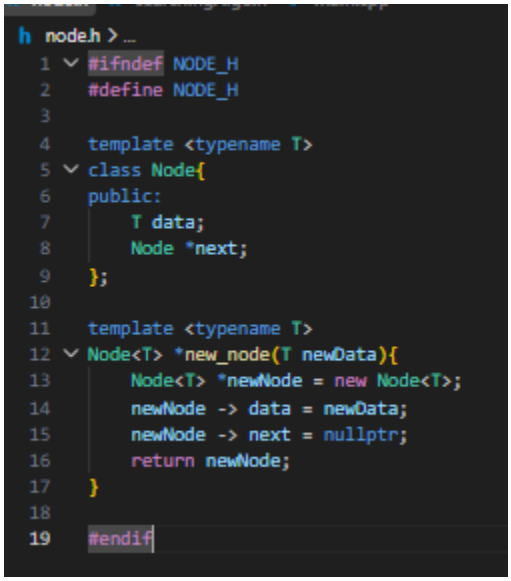
| | |
|---|---|
| | **node.h**<br>  <br>h node.h > ...<br>1 #ifndef NODE_H<br>2 #define NODE_H<br>3<br>4 template <typename T><br>5 class Node{<br>6 public:<br>7 T data;<br>8 Node *next;<br>9 };<br>10<br>11 template <typename T><br>12 Node<T> *new_node(T newData){<br>13 Node<T> *newNode = new Node<T>;<br>14 newNode -> data = newData;<br>15 newNode -> next = nullptr;<br>16 return newNode;<br>17 }<br>18<br>19 #endif |
| **Output** | PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 6><br>The data: P was found successfully. |
| **Observations** | I constructed a single linked list containing character nodes P R I N C E and implemented linearSearchLL to traverse the list starting from the head node. Searching for the character P resulted in an immediate match at the first node, terminating the traversal and outputting the data: P was found successfully. |

Table 6-3a. Binary Search for Arrays

| | |
|---|---|
| **Screenshot** | **main.cpp**<br><br>`//generate a sorted array: (use for loop)`<br>`int newArr[10] = {1,2,3,4,5,6,7,8,9,10};`<br><br>`for (int i = 0; i < max_size; i++ ){`<br>`  newArr[i] = i;`<br>`}`<br><br>`binarySearchArr(newArr, max_size, 100);`<br><br>**searchingAlgo.h**<br><br>`template<typename T>`<br>`void binarySearchArr(T data[], int arrSize, T dataFind){`<br>`  int low = 0, high = arrSize - 1, middle;`<br>`  while(low <= high){`<br>`  middle = (int)((low + high/2));`<br>`  if(dataFind == data[middle]){`<br>`  std::cout<<"The data was found\n";`<br>`  return;`<br>`  }`<br>`  else if(dataFind < data[middle]){`<br>`  high = middle - 1;`<br>`  }`<br>`  else{`<br>`  low = middle + 1;`<br>`  }`<br>`  }`<br><br>`  std::cout<<"The data was not found\n";`<br>`}` |
| **Output** | `PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 6>`<br>`The data was not found` |
| **Observations** | I implemented an iterative binary search function on a sorted array populated from 0 to 9 using a for loop. When searching for the value 100, the algorithm repeatedly halved the search space by updating the boundary indices until low exceeded high. |

Table 6-3b. Binary Search for Linked List

**Screenshot**

main.cpp

```
Node<int> *char1 = new_node(10);
Node<int> *char2 = new_node(11);
Node<int> *char3 = new_node(12);
Node<int> *char4 = new_node(13);
Node<int> *char5 = new_node(14);
Node<int> *char6 = new_node(15);
Node<int> *char7 = new_node(16);

// Connect the nodes
char1->next = char2;
char2->next = char3;
char3->next = char4;
char4->next = char5;
char5->next = char6;
char6->next = char7;
char7->next = nullptr;

binarySearchLL(char1, 18);

return 0;
```

**searchingAlgo.h**

```
//middle function getter:
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){
    Node<T>* slow = start;
    Node<T>* fast = start;

    //continue until fast reaches the end or the last node of the current search rang

    while(fast != last && fast->next != last){
        //move slow one step
        slow = slow->next;
        //move fast two steps
        fast = fast->next->next;
    }
    return slow; // return middle
}

template<typename T>
void binarySearchLL(Node<T>* start, T dataFind){
    //step 1: Set start to the head of the list and set last to null
    Node<T>* last = nullptr;

    // Middle element is calculated using the two pointers approach discussed above
    while(start != last){
        Node<T> *middle = getMiddle(start, last);

        //If the middle element is same as the key to be sorted, we return it
        if(middle->data == dataFind){
            std::cout<<"searching is Successfull\n";
            return;
        }
        else if(middle->data > dataFind){
            last = middle;
        }
        else{
            start = middle->next;
        }
    }
    std::cout<<"Searching is unsuccessful\n";
}

#endif
```

| | |
|---|---|
| **Output** |  |
| **Observations** | I constructed a sorted linked list with integer values ranging from 10 to 16 using a custom node structure. By calling binarySearchLL with the target value 18, the algorithm repeatedly located the middle node using slow and fast pointers and adjusted the search boundaries. |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1

**Screenshot**

problem1.cpp > ...

```cpp
#include<iostream>
#include "node.h"

//array sequential search
int linearSearchArray(int arr[], int size, int key) {
    int comparisons = 0;
    for(int i = 0; i < size; i++){
        comparisons++;
        if(arr[i] == key) return comparisons;
    }
    return comparisons;
}

//linked list sequential search
int linearSearchLinkedList(Node<int>*head, int key) {
    int comparisons = 0;
    while (head != nullptr) {
        comparisons++;
        if(head->data == key) return comparisons;
        head = head->next;
    }
    return comparisons;
}

int main(){
    int dataset[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int key = 18;

    //array search
    int arrComp = linearSearchArray(dataset, 10, key);
    std::cout << "Array comparisons: " << arrComp << std::endl;

    //build linked list loop
    Node<int>* head = new_node(dataset[0]);
    Node<int>* curr = head;
    for (int i = 1; i < 10; i++ ) {
        curr->next = new_node(dataset[i]);
        curr = curr->next;
    }
    //linked list search
    int llcomp = linearSearchLinkedList(head, key);
    std::cout << "Linked List Comparisons: " << llcomp << std::endl;
}
```

| | |
|---|---|
| **Output** | PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 6><br>Array comparisons: 2<br>Linked List Comparisons: 2 |
| **Observations** | In this first problem, I implemented a sequential search on both an array and a linked list to locate the key 18. I observed that both approaches required exactly two comparisons because the target element appeared at the second position in the dataset. |

Problem 2

**Screenshot**

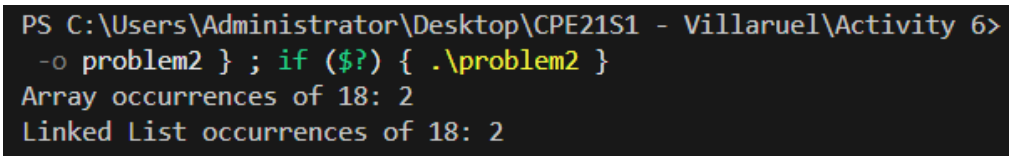
problem2.cpp > main()

```cpp
#include<iostream>
#include "node.h"

//count occurrences in array
int countOccurrenceArray(int arr[], int size, int key) {
    int count = 0;
    for(int i = 0; i < size; i++ ){
        if (arr[i] == key){
            count++;
        }
    }
    return count;
}
//count occurrence in linked list
int countOccurrenceLL(Node<int>*head, int key) {
    int count = 0;
    while (head != nullptr) {
        if (head->data == key){
            count++;
        }
        head = head->next;
    }
    return count;
}
int main() {
    int dataset[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;
    int k = 18;
    //array count
    int arrCount = countOccurrenceArray(dataset, size, k);
    std::cout<<"Array occurrences of "<< k << ": "<< arrCount << std::endl;

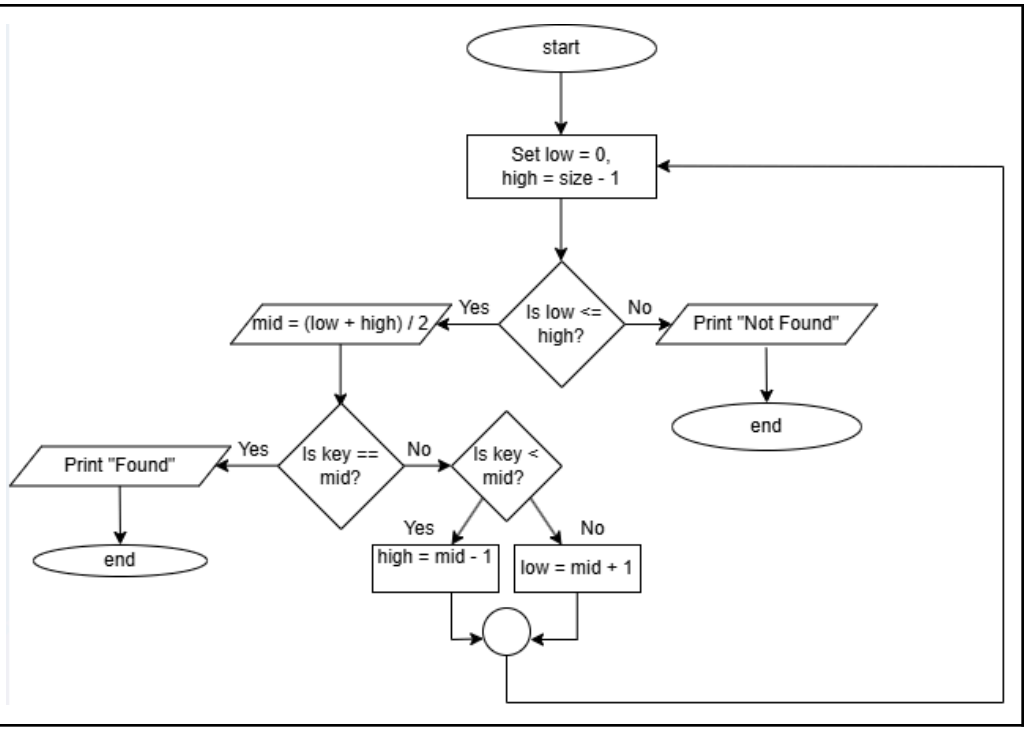
Node<int>* head = new_node(dataset[0]);
    Node<int>* curr = head;
    for (int i = 1; i < 10; i++ ) {
        curr->next = new_node(dataset[i]);
        curr = curr->next;
    }

int llcount = countOccurrenceLL(head, k);
std::cout << "Linked List occurrences of " << k << ": " << llcount << std::endl;
return 0;
}
```

| | |
|---|---|
| **Output** |  |
| **Observations** | I modified the sequential search algorithm to traverse the entire dataset and count every occurrence of a given key rather than stopping at the first match. Testing this on the dataset allowed me to confirm that key 18 appeared twice across both the array and linked list structures. |

Problem 3

| | |
|---|---|
| **Algorithm** | **Binary Search - Array**<br>1. Set low = 0 and high = size - 1<br>2. While low <= high:<br>3. Calculate mid = (low + high) / 2<br>4. If arr[mid] == key, return true (found)<br>5. If key < arr[mid], set high = mid - 1<br>6. Else, set low = mid +1<br>7. Return false (not found)<br>**Binary Search - LL**<br>1. Set start = head and last = nullptr<br>2. While start != last:<br>3. Find the middle node<br>4. If middle->data == key, return true (found)<br>5. If middle->data > key, set last = middle<br>6. Else set start = middle->next. |

| | |
|---|---|
| **Flowchart** |  |

start
Set low = 0,
high = size - 1
Is low <= high?
Yes
No
mid = (low + high) / 2
Print "Not Found"
end
Is key == mid?
Yes
No
Print "Found"
end
Is key < mid?
Yes
No
high = mid - 1
low = mid + 1

**Screenshot**

problem3.cpp > getMiddle<T>(Node<T>*, Node<T>*)

```cpp
#include<iostream>
#include "node.h"

//get middle node
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last) {
    Node<T>* slow = start;
    Node<T>* fast = start;

    while (fast != last && fast->next != last) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
    //binary search for ll
    template<typename T>
    bool binarySearchLL(Node<T>* start, T dataFind) {
        Node<T>* last = nullptr;
        while (start != last) {
            Node<T>* middle = getMiddle(start, last);
            if (middle->data == dataFind) return true;
            else if (middle->data > dataFind) last = middle;
            else start = middle->next;
        }
        return false;
    }

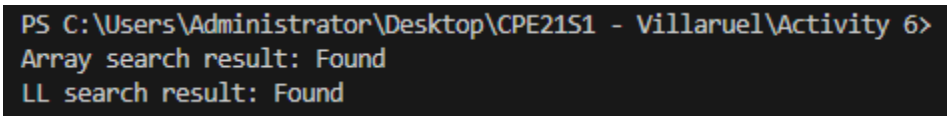
    //binary search array
    bool binarySearchArr(int arr[], int size, int key){
        int low = 0, high = size - 1;
        while (low <= high){
            int mid = (low + high) / 2;
            if (key == arr[mid]) return true;
            else if (key < arr[mid]) high = mid -1;
            else low = mid + 1;
        }
        return false;
    }

    int main() {
    int dataset[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int size = 10;
    int key = 8;

    //array binary search
    bool foundArr = binarySearchArr(dataset, size, key);
    std::cout << "Array search result: "<<(foundArr ? "Found" : "Not Found") << std::endl;

    Node<int>* head = new_node(dataset[0]);
    Node<int>* curr = head;
    for (int i = 1; i < 10; i++ ) {
        curr->next = new_node(dataset[i]);
        curr = curr->next;
    }

    //linked list binary search
    bool foundLL = binarySearchLL(head, key);
    std::cout << "LL search result: "<<(foundLL ? "Found" : "Not Found") << std::endl;
    return 0;
    }
```

| | |
|---|---|
| **Output** |  |
| **Observations** | I implemented an iterative binary search on a sorted array and linked list to trace how the search space is divided in each step. By tracking the middle elements. I verified that finding key 8 required four iterations as the search range was repeatedly halved. |

Problem 4

**Screenshot**

problem4.cpp > main()

```cpp
#include<iostream>
#include "node.h"

//recursive binary search array
int recursiveBinarySearchArr(int arr[], int low, int high, int key) {
    if(low > high) return -1;

    int mid = (low + high) / 2;
    if (arr[mid] == key) return mid;
    else if (arr[mid] > key) return recursiveBinarySearchArr(arr, low, mid - 1, key);
    else return recursiveBinarySearchArr(arr, mid + 1, high, key);
}

//get middle node for ll binary search
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last) {
    Node<T>* slow = start;
    Node<T>* fast = start;

    while (fast != last && fast->next != last) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

//recursive binary search for ll
template <typename T>
bool recursiveBinarySearchLL(Node<T>* start, Node<T>* last, T key) {
    if (start == last) return false;

    Node<T>* middle = getMiddle(start, last);
    if (middle->data == key) return true;
    else if (middle->data > key) return recursiveBinarySearchLL(start, middle, key);
    else return recursiveBinarySearchLL(middle->next, last, key);
}

int main(){
    int dataset[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int size = 10;
    int key = 8;

    //array recusrive searcvh
    int index = recursiveBinarySearchArr(dataset, 0, size - 1, key);
    std::cout << "Array search result: " << (index != -1 ? "Found" : "Not Found") << std::endl;

    Node<int>* head = new_node(dataset[0]);
    Node<int>* curr = head;
    for (int i = 1; i < 10; i++ ) {
        curr->next = new_node(dataset[i]);
        curr = curr->next;
    }

    //ll recyrsive search
    bool foundLL = recursiveBinarySearchLL(head, (Node<int>*)nullptr, key);
    std::cout << "LL search result: " << (foundLL ? "Found" : "Not Found") << std::endl;
    return 0;
}
```

**Output**

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Activity 6>
Array search result: Found
LL search result: Found
```

**Observations**

I converted the binary search algorithm into a recursive function for both data structures. I

| | |
|---|---|
| | observed that using recursion produced cleaner code while achieving the exact same search steps and results as the iterative method. |

**Conclusion**

Through completing this activity on search algorithms, I gained a deep practical understanding of how linear and binary search techniques operate on both arrays and linked lists, as well as the fundamental differences between iterative and recursive approaches. However, I encountered significant challenges throughout the process, particularly with debugging hidden non breaking space characters that caused compilation errors. Overcoming these obstacles ultimately improved in C++ programming skills and reinforced the importance of writing clean syntax and choosing the appropriate data structure for efficient data retrieval.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Google Gemini and Claude**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Code assistance and explanation of concepts**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **For this activity, I used google gemini to explain the concepts so I could analyze my work further, and claude for coding assistance to fix errors and help me understand the code.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.